Proofs for Now and Proofs for Later

The Amortization Horizon of Human and Artificial Reasoning in Lean

Working paper

2 August 2026

Abstract

A checked proof establishes that a proposition follows, but not what future it was built to serve. We propose the *amortization horizon*: the range over which a constructor expects an abstraction to repay its cost. A proof produces a kernel certificate, working memory, and an interface for later users. In 3,630 pairs of validated Lean proofs of identical statements, AI-provenance scripts create more local claims, yet each receives fewer downstream references, is almost threefold less likely to state a callable family, and is more generically named. Elaboration more often erases or duplicates AI boundaries; human boundaries more often map one-to-one. Neither conditional source duration nor aggregate multi-use sharing robustly favors humans. This consolidation–composition split locates the divergence in interface selection. Certificate topology also fails consumer-invariance: Lean versions encode the same source claim differently, while extensionally equivalent representatives can differ exponentially in tree size. An exploratory model finds a short information pulse at human claim boundaries, attenuated at AI boundaries. The divergence lies not in deductive power or biological kind, but in which future consumer construction serves. We formalize the principle as library-amortized proof complexity over theorem streams, a quantity single-theorem benchmarks cannot observe.

1 The missing future of a checked proof

A theorem prover settles a sharply posed question: did this term check at this type? That question is essential, but it is not the whole function of proof. A mathematician also uses a proof to find the invariant that makes a result intelligible, to remember a route through an argument, to expose a lemma that will solve the next theorem, and to leave an object another person can repair (Rav, 1999). These uses extend beyond the instant of verification. Two certificates of the same statement may thus agree perfectly as kernel inputs while serving different futures.

This paper argues that this future is the most useful level at which to compare present human- and AI-provenance Lean proofs. The common contrast—human proof versus machine proof—encourages an essentialist question: what signature does the identity of the constructor leave in a proof graph? That question is poorly controlled. Theorem, library, tactic, model, and representation all change graph statistics. More importantly, it mistakes an objective for an organism. Most generated proofs are rewarded when the current declaration compiles. Much human formalization occurs inside a longer activity of exposition, maintenance, and library construction; early workflow evidence also finds that users want AI assistance while preserving high-level control (Collins et al., 2026). Different artifacts are therefore rational even before any difference in cognitive machinery is posited.

We call the relevant variable the *amortization horizon*: how far into future reasoning the constructor expects a present abstraction to pay. At a short horizon, a local claim can be useful merely because it records a search result for the next tactic. Its name need only be distinct; its statement need not be reusable; it may disappear under elaboration. At a long horizon, names, boundaries, generality, and stable dependencies become investments. Their up-front cost can be recovered across later steps, theorems, versions, and readers.

This account makes four contributions. First, it separates three products of formal construction: certificate, episode structure, and interface. Second, it tests their separation in 3,630 exact- statement human–AI pairs, with a scope-aware decoding of the dataset's production proof terms and a two-version representation audit. Third, it offers an exploratory information-theoretic test inspired by work on how speakers regulate surprise in conversation (Bergey and DeDeo, 2024). Finally, it proposes a sequence-level object, amortized proof complexity,

that charges for a library and the stream of proofs it enables. This reframes a central evaluation problem: success at one theorem measures certificate production, not mathematical accumulation.

The argument deliberately narrows several earlier claims in this repository. Viteri and DeDeo's proof-network work establishes that proof organization can support robust belief and reveals heavy-tailed reuse (Viteri and DeDeo, 2022). Our replication found those results informative but highly sensitive to the graph boundary. Near-saturated belief did not distinguish paired certificates, and a newly discovered de Bruijn-scope conflation makes the inherited unexpanded-term topology unsuitable as a semantic reuse measure. Rather than force authorship into those statistics, we use the failure to identify the missing object: who or what must use the proof next?

2 A proof has three consumers

The word "proof" compresses at least three artifacts.

Certificate. The kernel consumes an elaborated term and checks that it inhabits the target type. The certificate may contain local `let` binders and shared subexpressions, but the kernel does not require human-readable names, explanatory order, or a useful public API.

Episode structure. The constructor and an immediate reader consume the source script during the current attempt. A named `have` makes a fact addressable, reduces the active goal, and records search state. It can perform this role even if no later source token names it: a contextual tactic such as `linarith` may consume it implicitly, or elaboration may erase it. In cognitive terms this is a distributed representation: internal and external state jointly carry the task, and writing a claim can be an epistemic action that lowers later computation (Zhang and Norman, 1994; Kirsh and Maglio, 1994).

Interface. Future proofs and maintainers consume exported declarations, stable names, general statements, and explanatory boundaries. An interface is not just a set of dependencies. It is a retrieval scheme: it partitions a derivation into units that can be recognized and invoked without reconstructing their interiors. This audience-relative view accords with the dialogical conception of proof as discourse directed toward explanatory persuasion (Dutilh Novaes, 2018). DeDeo and Duede call attention to the associated correspondence problem: a formal derivation can exist while failing to track the proof idea that is supposed to justify its relevance (DeDeo and Duede, 2026). The interface can also certify something about its constructor: explanatory choices provide evidence of understanding that bare correctness does not. Cheap generation can sever this costly-signal function even while improving verification (Wojtowicz and DeDeo, 2025). For this meta-consumer, construction cost is not mere overhead but part of the evidence; automation may preserve the visible artifact while changing what its production warrants.

These consumers define different notions of reuse. A proof-term binder used twice is a sharing node in a directed acyclic graph. A source claim referenced after several later claims is working memory with temporal reach. An exported theorem cited by later files is persistent memory. None is a universal measure of modularity. In particular, raw occurrence counts can mistake tactic expansion for conceptual importance, while a theorem that is used only once may still identify the right explanatory cut.

This separation imposes a methodological constraint. Let $R_c(e)$ be consumer c's response to artifact e, and write $e \sim_c e'$ when $R_c(e) = R_c(e')$. The *consumer-invariance principle* says that a statistic offered as a functional property for c must be constant on each $\sim_c$ class. For correctness, Lean's kernel accepts any term of the target type; proof irrelevance makes proofs of one proposition definitionally interchangeable, and zeta reduction replaces a `let`-bound variable by its value (Lean Developers, 2026). Consequently, binder count, outdegree, and even proof-tree size are not intrinsic properties of the established judgment. They can still diagnose checking cost, an elaborator snapshot, or a consumer that reads that representation, but the observable and representative must be named. There is a simple impossibility corollary. Fix a proposition and let the consumer's only response be kernel acceptance. All accepted proof terms then occupy one response class, so every consumer-invariant functional statistic is constant across them. Any nonconstant human–AI certificate statistic necessarily describes a richer response—time, memory, elaboration, inspection, retrieval, or provenance—rather than correctness itself. This is the proof-theoretic identity problem in empirical form: normalization and generality induce different

quotients over derivations (Došen, 2004), and an authorship statistic must declare which quotient, if any, it survives.

The failure can be exponential, not cosmetic. Let e_0 be the natural-number literal zero and $e_{n+1} =$ `let x :=` e_n`; x + x`. Retaining sharing gives $1+7n$ syntax-tree nodes in our Lean encoding; full zeta reduction gives $4 \cdot 2^n - 3$. At $n = 14$, Lean's own reducer changes 99 nodes and 14 binders into 65,533 nodes and none, without changing what conversion can establish. Thus a proof graph is a representative of checked evidence, not a representation-free map of reasoning. Elaboration is better treated as a two-stage channel: first ask whether a visible source boundary is retained at all, then, conditional on retention, whether it is used zero, once, or many times. The empirical toolchain comparison below shows why those stages cannot be collapsed.

For an abstraction z and consumer c, a minimal decision model is

$$V_{\gamma,c}(z) = \sum_{u \in U_c(z)} \gamma^{d(z,u)} s_c(u) - c_{\text{make}}(z) - c_{\text{interface},c}(z), \tag{1}$$

where $U_c(z)$ is the set of anticipated uses, $s_c(u)$ is work saved at use u, and d is distance in steps, theorems, time, or agents. The discount γ summarizes the construction horizon. Interface work includes choosing a statement at the right generality, selecting a memorable name, documenting it, controlling imports, and maintaining it across versions. These costs can dominate when the only scored outcome is today's compilation. A short-horizon system may therefore be globally wasteful while locally optimal.

The consumer index matters as much as the discount. The same lemma can have positive value for a retrieval-equipped prover and negative value for a reader who cannot recognize or trust it. There is consequently no consumer-free scalar called "lemma quality." Horizon specifies when value must be recovered; audience specifies whose future work counts.

This is also a rate–distortion problem. Let $E \sim p(e)$ denote completed proof episodes and Z the interfaces retained from them. For consumer c, define $D_c(E, Z)$ as the expected extra work, error, or loss of correspondence when future tasks receive Z rather than the full episode. An interface policy trades representational rate against discounted distortion, for example by minimizing $I(E; Z) + \beta \mathbb{E}_\gamma[D_c(E, Z)]$ (Shannon, 1959). Consumer-invariance identifies the zero-distortion quotient; horizon and counterfactual width determine the future-task distribution. The same derivation can therefore have different optimal compressions for a kernel, a retrieval-equipped prover, and a person tracking the proof idea.

Distance is only one dimension of the wager. A claim can also range over counterfactual cases that the current derivation never visits. A parameterized lemma increases this *counterfactual width*: it packages a family of related facts rather than only the instance presently needed. Horizon measures how far an abstraction travels; width measures how many varying contexts it can enter. This echoes the distinction between instance-based and generic explanation (Linnebo, 2022). Both enlarge an abstraction's amortization domain, and both cost something before they save work.

The principle yields discriminating predictions. Holding the theorem fixed, a shorter horizon should produce more local claims that are unused or single-use, less semantic investment in names, and shorter visible spans between definition and last reference. It does *not* predict that every machine claim is unreused, nor that every human claim is explanatory. Most importantly, it predicts an intervention: an AI given downstream library and maintenance objectives should cross the observed divide. Recent systems that explicitly extract reusable theorems or refactor generated proofs provide early evidence for exactly this mobility (Zhou et al., 2024; Fu et al., 2026; Lu et al., 2026; Lyu et al., 2026).

3 Paired observation at three resolutions

3.1 Corpus and estimand

We use the local NuminaMath-LEAN proof-artifact shards (AI-MO, 2025). A record is retained when human and prover proofs are both available and marked valid, the human ground-truth annotation is complete, and the UUID is unique. We then require both serialized artifacts to contain the same normalized final declaration signature. This audit removes one nominally valid prover artifact with no target and four mismatches: one wrong theorem join, one helper-only output, and two altered propositions. The result is 3,630 exact-statement pairs from 12 coarse source groups. We analyze only the final target declaration. In 158 pairs, both tracks contain identical pre-target declarations after whitespace and comment normalization; 11 additional AI artifacts

introduce helpers named only `lemma_1` or `lemma_2`, seven of which the target cites. Pooling these declarations would therefore mix shared problem scaffolding and side-specific output with the target-proof comparison, not reveal a human-only miniature library.

The dataset card defines its formal ground truth as a proof written by a human annotator and its prover field as a model proof produced during reinforcement-learning training (AI-MO, 2025). "Human" and "AI" below denote those artifact fields. This does not mean unaided human cognition versus an isolated model. Selection, shared libraries, model training data, and annotation workflows remain possible causes. Pairing removes theorem identity, not those process differences. Our estimand is consequently descriptive: how these two artifact tracks differ under the workflows that produced them.

At source level, a Unicode-aware parser removes comments and strings, locates named `have` claims in the final proof body, and counts later exact-name references. The counting window stops at same-name redeclaration, a conservative guard against shadowing. A claim has *long visible reach* when its last reference occurs after at least one intervening claim boundary. Names matching generic forms such as `h`, `h7`, `this`, `step2`, and `aux` are classified as placeholders. A family claim has either binders before its type or a type beginning with `forall`/∀ (up to parentheses), so equivalent Lean spellings enter the same class. Because boundary count depends on how densely a script is decomposed, we separate adoption from duration and also report token distance and the fraction of available later boundaries crossed. Results are pooled at claim level, with 95% intervals from resampling the 12 source groups. Pairwise proof metrics use Wilcoxon signed-rank tests; the effect sizes, source consistency, and sensitivity checks matter more than the very small p-values.

Lexical reference is not semantic use. We therefore elaborate all 7,260 target artifacts against the dataset's production snapshot, Lean/Mathlib 4.15.0. That version represents a source `have` as an application of `letFun` to a named lambda; Lean 4.33 instead uses a nondependent `letE`. A node-kind-only extractor would consequently report near-total erasure under one version. Our stack-safe decoder recognizes both semantic encodings. A de-Bruijn-aware traversal counts binder occurrences while increasing depth under nested binders, and exact-name longest common subsequence in preorder aligns source claims to decoded binders. A stack-safe dynamic program preserves full tree-occurrence counts while memoizing shared subexpressions; every status and code/toolchain hash is serialized. All 7,260 tasks complete.

As a representation sensitivity, we independently retain the preselected 312-pair scope-audit sample under Lean/Mathlib 4.33.0-rc1. Of 624 tasks, 604 compile and 298 pairs have both sides. Its 20 failures measure version attrition rather than production-time invalidity. Cross-version claims are compared only where both tasks compile and the source binder aligns under both decoders.

Finally, we score the 624 source documents in the preselected sample with a local `Q4_K_M` quantization of Goedel-Prover-V2-8B (Lin et al., 2025). Each token receives negative log probability given up to 8,192 preceding tokens. At each `have` boundary we compare the next 4, 8, 16, or 32 tokens with the preceding window and with eligible non-boundary locations in the same document. This is model-relative surprise under a prover trained on Lean-like text, not a measurement of human cognitive surprisal. Training overlap and affinity for the prover track are unexcluded. We therefore treat it as an exploratory timing assay rather than evidence of understanding.

3.2 Local claims have different fates

The paired proofs are not radically different in raw size: median final-body length is 125 whitespace tokens for the human track and 145 for AI, with a median within-pair increase of five tokens (source-cluster interval 2–15). They decompose that work differently. The median number of named claims is one versus four; pooled density is 2.02 human and 3.18 AI claims per 100 source tokens.

The important contrast is what happens after introduction (figure 1). Human claims receive 1.44 explicit references on average (source-cluster interval 1.34–1.59), compared with 0.87 for AI (0.83–0.89). Of human claims, 22.2% have zero visible uptake and 26.5% have multiple references. For AI the corresponding values are 36.3% and 12.1%. All 12 source groups show lower references per AI claim and higher zero-uptake share. Human claims also more often remain live across a later claim boundary: 48.4% versus 41.8%, in the human direction for 11 of 12 groups. The result is not simply that AI proofs are longer; it concerns the conditional fate of an introduced abstraction. In 834 pairs within 10% in body length, uses remain 1.35 versus 1.06, zero uptake 24.8% versus 33.4%, and long reach 49.4% versus 43.4%. Reach has an extensive and an intensive margin. Human claims are explicitly adopted 77.7% of the time versus 63.7% for AI; conditional on adoption, they receive 1.86 versus 1.36 explicit calls, and their last reference lies 56.8 versus 43.1 source tokens away. Counts of

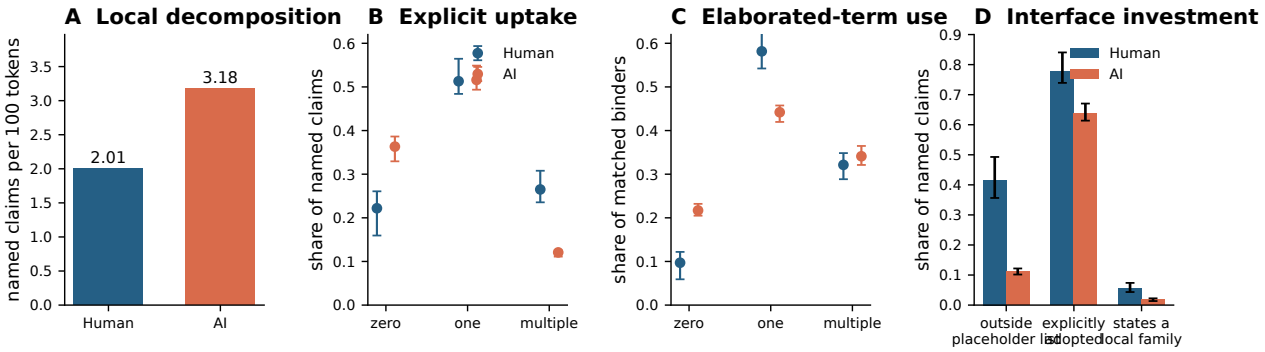

Figure 1: Three consumers of local claims. Source results use all 3,630 theorem pairs; term results use the pairs that compiled on both sides. Error bars in panels B–D are 95% source-cluster intervals. "Outside placeholder" means a name outside the conservative generic-form list; "explicitly adopted" means at least one later exact-name reference; "local family" collapses binder and explicit-universal spellings.

intervening claims are endogenous to decomposition density: given adoption and at least one available later boundary, the probability of crossing one is indistinguishable (75.1% versus 75.2%). Human claims instead cross a larger fraction of the available later boundaries (49.1% versus 45.4%). We therefore interpret the aggregate difference as lower AI adoption, not a representation-free lifetime constant. Indeed, within 834 length-matched pairs neither conditional-duration measure separates the tracks; in 210 pairs with the same positive claim count, normalized duration reverses (51.1% versus 53.6%). The robust source divergence is explicit adoption, not how long an adopted claim lives. Nor is it arithmetic from emitting more claims. In 210 pairs with the same positive claim count, explicit uses per claim remain 1.39 versus 1.11 (AI-minus-human source-cluster interval −0.428 to −0.182), and zero uptake remains 21.8% versus 29.7% (0.050 to 0.107). Nor does repeated generic naming create the result: among names declared only once per proof, uses are 1.54 versus 0.92, zero uptake 19.1% versus 33.2%, and long reach 52.5% versus 46.0%.

The strongest direct signature of counterfactual width is grammatical. A local lemma can turn one fact into a callable family either with binders—`have sum_range (n : Nat) : ...`—or with an explicitly universal type, `have sum_range : forall n, ...`. Counting only the first spelling produces a dramatic but non-invariant fourteenfold contrast. Quotienting the two forms still leaves a large difference: 835 of 14,911 human claims (5.60%) and 491 of 25,715 AI claims (1.91%) state a family, with the human direction in all 12 source groups. Within identical-theorem pairs, a generalized claim occurs only on the human side in 320 cases, only on the AI side in 108, and on both in 171. This remains a syntactic classifier, not proof that a statement chose the best generality. But it records a real construction cost: making an intermediate result vary over an argument. Conditional on paying that cost, AI families are not shorter-lived by this pooled measure: 72.0% of the human and 77.2% of the AI generalized claims remain live across a later claim boundary. Here too the difference lies in how often the abstraction is selected, not an inability to use it once made. The classifier has the same functional correlate in both tracks. Minimum-cost matching to nearby instance claims, with a quarter-proof position caliper, retains 269 human and 226 AI proofs: family claims are respectively 10.2 and 15.8 points more likely to be adopted, and 20.8 and 31.0 points more likely to be multiply referenced; all four intervals exclude zero. The association is not explained by families occurring earlier. It remains descriptive, but the rarer AI families are not inert when selected. In the 210 pairs with the same positive claim count, family shares remain 4.48% versus 2.47%. The difference is not carried by closed-goal computation: after excluding 416 pairs in which either side invokes `native_decide`, family shares remain 5.35% versus 1.92%.

Names show an even larger difference. Placeholder forms account for 58.4% of human claims and 88.8% of AI claims. The pooled human name distribution has 3,716 types, Shannon entropy 8.58 bits, and effective vocabulary 384; the AI distribution has 1,710 types, 6.28 bits, and effective vocabulary 78 despite containing 72% more name tokens. Length is a crude but convergent check: 24.1% of human names and 9.4% of AI names have at least four characters. This is consistent with interface investment, but it is not by itself proof of semantic quality. A short conventional name may be excellent, and a long name may be decorative. A paired within-proof embedding assay finds no robust human advantage after chance and claim-count normalization (appendix), so we make no semantic naming claim. Nor do the proxies collapse into one style score. Across the 1,885 pairs

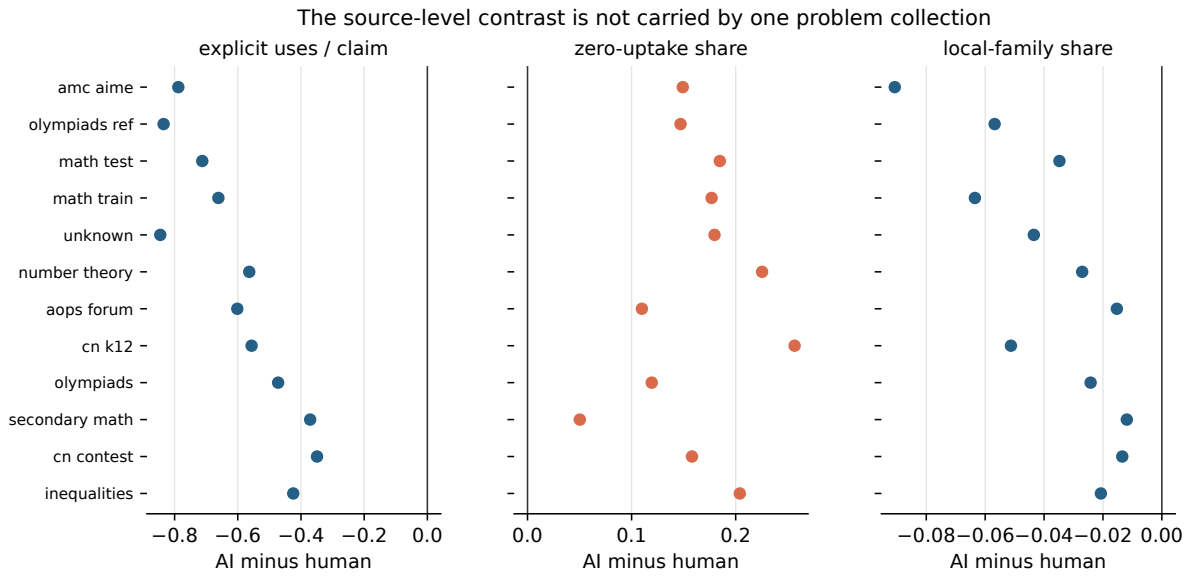

Figure 2: Source-group consistency. Each point is one of 12 source groups and shows AI minus human. References per claim are lower for AI in every group; zero-uptake is higher in every group; local family share is lower in every group. Point sizes encode paired sample count.

with claims on both sides, Spearman correlations among proof-level human advantages in uptake, reach, family construction, and naming range only from .07 to .32. We therefore keep the coordinates separate.

The elaborated term analysis both confirms and corrects the source story. All 7,260 production- snapshot tasks compile; exact-name alignment retains 14,832 of 14,911 human and 25,677 of 25,715 AI claims. Among aligned binders, 9.7% human versus 21.7% AI have zero occurrence in the final term. But multiple occurrence is 32.1% versus 34.1% (AI-minus-human interval −0.05 to 4.89 points). The sharper result is a polarized source-to-certificate channel: zero-or-multiple occurrence is 41.8% human and 55.8% AI (difference 14.0 points, interval 9.9–20.6), leaving one-to-one mapping at 58.2% versus 44.2%. Within both the source-single-use and source-multi-use strata, AI claims are more likely to disappear and more likely to be duplicated; polarization is higher for AI in all 12 matched tactic strata. Contextual tactics can amplify a fact massively, so raw means are not cognitive reuse measures. What survives is more precise: AI source boundaries align less often one-to-one with certificate structure, while aggregate kernel multi-use is not uniquely human.

The three resolutions reveal a *consolidation–composition split*. The human track more often turns intermediate state into a visibly adopted, repeatedly invoked, named, and general interface; the AI track more often leaves its bound fact absent from or amplified within the returned certificate. Yet conditional duration is control-dependent, AI families are at least as long-reaching in the pooled comparison, and retained term binders are just as often multiply used. The present divergence is therefore better localized to consolidation policy—where compositional structure is made explicit—than to capacity for composing in a checked term.

3.3 One theorem, two amortization horizons

An extreme pair makes the distinction legible. Define an integer sequence by $f(0) = 2001$, $f(1) = 2002$, $f(2) = 2003$, and $f(n+3) = f(n) + f(n+1) - f(n+2)$; the target asks only for $f(2003) = 0$. The human-provenance script proves by induction a simultaneous family

$$f(2k+2) = 2003 + 2k, \qquad f(2k+1) = 2002 - 2k, \qquad f(2k) = 2001 + 2k,$$

names it `f_formula`, and instantiates $k = 1001$. The AI script is essentially one command: `norm_num [f]`. One proof invests in a counterfactually wide description; the other asks the elaborator to settle the present instance.

Both check, but their representation costs reveal why certificate topology needs a named consumer. The human term has 278,395 nodes under full tree-occurrence accounting. Repeated expansion of the AI term has 5.68×10^{534} nodes—a 535-digit count—although shared expression structure lets Lean store and check it

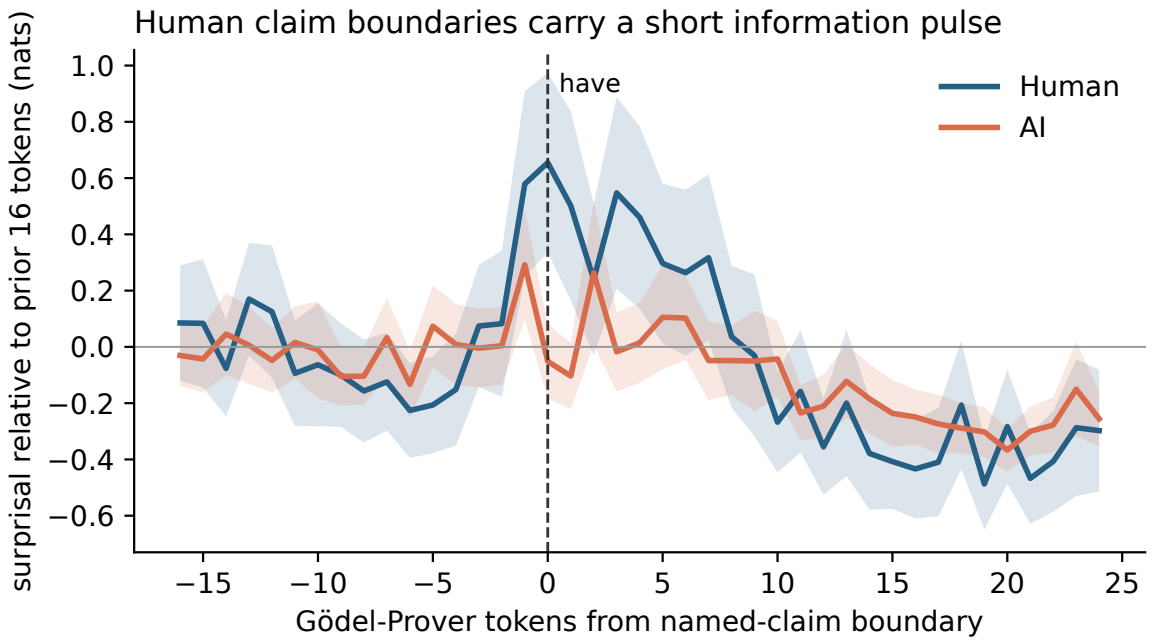

Figure 3: Exploratory model-relative information around source `have` boundaries. Curves show negative log probability relative to the preceding 16 tokens; bands are proof-weighted 95% standard-error intervals. The model is Goedel-Prover-V2-8B in raw-code mode. The result may reflect training overlap, naming conventions, or prover-style affinity and is not a human surprisal measurement.

without materializing that tree. Two other AI `norm_num` certificates have 281- and 427-digit expanded sizes; the largest human count has 11 digits. These numbers are not a cognitive score. They show a real compile-first route in which a tiny source episode denotes an astronomical repeated computation, while the human route identifies a reusable law. In complexity language, sharing compresses a formula into a circuit, but the human step does something different again: it changes the algorithmic description from instance expansion to an inductive invariant. The pair was selected for clarity; the aggregate family and uptake contrasts were fixed independently, and reverse examples exist.

3.4 Claim boundaries carry a short information pulse

Human conversational production regulates information in time: pauses and backchannels appear near changes in information rate (Bergey and DeDeo, 2024). If a named claim is an explanatory boundary rather than only syntax, it might likewise concentrate material that a predictive model finds locally unexpected.

The exploratory result is compatible with that prediction (figure 3). Across the 177 pairs in which both proofs contain eligible boundaries, the median eight-token boundary excess— post-boundary minus pre-boundary NLL, corrected by same-document control positions—is 0.357 nats per token for human scripts and 0.028 for AI. The paired difference is -0.236 ($p = 8.9 \times 10^{-6}$). Removing the complete `have` header leaves an excess of 0.207 versus 0.044 ($p = .009$). The effect is present in 4-, 8-, and more weakly 16-token windows, and disappears by 32 tokens. Thus it is a local pulse, not sustained global difficulty.

The model assigns AI documents much lower NLL overall (median 0.73 versus 1.15), a warning about style affinity or data overlap. A leave-one-source-out token bigram preserves the raw boundary direction ($p = .017$), but its content-only contrast is marginal ($p = .051$). We infer neither creativity nor cognitive surprise. Nor does the pulse specifically validate the family classifier: within documents containing both kinds of claim, family and instance content pulses do not differ detectably (human $p = .62$; AI $p = .10$). At most, human boundaries mark a brief predictive discontinuity whose semantic interpretation remains model-dependent.

4 From provenance to objective

The horizon principle would be weak if it merely renamed the observed label. Its main content is that the difference should move when the objective moves. Several recent lines of work support this prediction. REFACTOR extracts heavily reused theorems from completed Metamath proofs (Zhou et al., 2024). Proof-Refactor and Lean Refactor explicitly optimize modularity, maintenance, checking cost, or version robustness after compilation (Fu et al., 2026; Lu et al., 2026). CircuitProver provides a closer intervention: accumulating verified lemmas and strategies across related Lean tasks reduces both proof length and checking time in ablation (Yang et al., 2026). Rtl2lean likewise asks a generator to build a layered reusable library (Lyu et al., 2026). At research-development scale, LeanMarathon makes an evolving formal and natural-language blueprint the shared state of a long-running autoformalization process; durable contracts and local repair support 258 proved declarations across its reported runs (Zhang et al., 2026b).

The ingredients have precedents: mined HOL Light inferences, LEGO-Prover, and DreamProver all turn intermediate results into later proving capacity (Kaliszyk and Urban, 2014; Wang et al., 2023; Zhang et al., 2026a). They also expose the retrieval constraint: persistent memory helps only if a later agent can select it; LeanSearch v2 reports a large gain from global premise retrieval (Gao et al., 2026). Our contribution is a common account of compile-first artifacts, refactored proofs, readable boundaries, and growing libraries, plus an objective that charges construction against future savings.

These systems do not prove that their artifacts match mature human libraries, and their metrics are not directly comparable with ours. They establish the more fundamental point: machines can be made to externalize persistent mathematical memory. The observed human–AI difference is therefore best read as a difference between present production regimes. The causal experiment now suggested is straightforward: assign the same base model the same theorem stream under (i) per-theorem compile reward and (ii) downstream cost, maintenance, and human-retrieval reward. Measure dead binders, claim reach, API survival, later citation, repair cost, and reader comprehension. If the horizons do not separate, our account fails.

The distinction also clarifies DeDeo's discussion of "alien lemmas." His argument draws on proof-complexity and average-case results to contrast hard-to-prove truths with a mathematical practice that seems to select truths with good reasons (DeDeo, 2024b). An AI may discover a lemma that dramatically compresses later certificates yet remains unrecognizable to human mathematical ontology. Such a lemma has high machine amortized value and low human interface value. It is neither simply good nor bad. It exposes two horizons with different consumers. The correspondence problem then becomes an engineering and epistemic demand: construct a bridge that preserves the machine compression while letting human readers track why the abstraction matters (DeDeo, 2024a; DeDeo and Duede, 2026).

5 Cognitive implications: abstraction as memory allocation

The source-level difference suggests a cognitive interpretation more specific than "humans are abstract" and more defensible than reading mental states directly from code. Constructing a proof requires allocating limited representational capacity. A solver can preserve the past by keeping facts in the local context, by naming a relation that summarizes many facts, or by retrieving a library theorem that packages an older derivation. These are memory operations at different scales. The horizon determines which compression is worth performing.

At the scale of one search episode, an intermediate claim can function like a checkpoint. It freezes a successful subcomputation, constrains the remaining goal, and provides the next tactic with an enlarged context. Generic labels are adequate because identity can be recovered from position. Dead elaborated binders are not necessarily mistakes on this account. They are traces of a search process whose utility was exhausted before the final certificate. Removing them may improve the artifact while making the original generation harder to understand, just as deleting a scratch calculation produces a cleaner solution but erases evidence about discovery.

At a longer scale, an abstraction is a lossy compression chosen for transfer. The statement forgets derivational detail while preserving a relation expected to matter again. A descriptive name supplies a retrieval cue; a proof boundary asserts that the interior can be temporarily ignored; a general signature enlarges the class of future contexts in which the package applies. This compression is not free. Choosing the wrong invariant can increase working-memory demand, and exporting too many lemmas makes the library harder to search. Long-horizon construction is therefore not maximization of lemma count. It is selection of a small basis whose elements remain useful after the local context has vanished.

The equality of multi-use term binders is theoretically important here. Kernel sharing and communicative abstraction are different achievements. Automation can expand a generic hypothesis into many term occurrences without ever identifying a concept for a reader. Conversely, a well-chosen explanatory lemma can deserve a name even when invoked once in the current theorem, because its intended reuse lies in another proof or another mind. Any cognitive account based only on the certificate DAG will confuse these cases. The units of human mathematical thought are not guaranteed to coincide with the compiler's units of common-subexpression elimination.

The local surprisal pulse offers a tentative temporal signature of this selection. A useful boundary often arrives when prior manipulations have made a new description possible: monotonicity, invariance, symmetry, or a change of representation. The words immediately after the boundary can therefore be hard to predict from nearby syntax even though the resulting claim makes the rest of the argument easier. This connects to an account on which explanations of necessary truths mark search-simplifying steps or sites of cognitive load (Kardeş and DeDeo, 2025). In conversational terms, the constructor briefly spends information to renegotiate common ground. The pulse's disappearance after 16 tokens is consistent with a boundary event rather than generally ornate prose. But this interpretation must remain provisional until human reading time, recall, and transfer are measured directly.

This perspective also distinguishes discovery from presentation without separating them absolutely. A source proof can be a search trace, a pedagogical artifact, or both. Humans often refactor scratch work into a proof idea; present generators are commonly evaluated before that second pass. The central divergence may thus be procedural: one workflow includes consolidation, naming, and anticipation of reuse, while another terminates at verification. A cognitively serious comparison should align stages—first attempts with first attempts, refactored artifacts with refactored artifacts—and observe how representations change between them. The horizon principle predicts that the largest human–AI gap will occur not in local inference, where both can exploit formal context, but in deciding what part of a successful episode should become shared memory.

6 Library-amortized proof complexity

Ordinary proof complexity asks for the size of a certificate for one formula in a fixed proof system (Cook and Reckhow, 1979; Krajíček, 1995). This leaves library construction outside the objective. Let $\Phi = (\varphi_1, \ldots, \varphi_T)$ be a stream of targets. Let L be an ordered, acyclic sequence of auxiliary declarations: each signature and body must check in the base environment plus earlier members of L, and $|L|$ charges both. If P_t checks φ_t in that extended environment and $|P_t \mid L|$ is its residual certificate size, define

$$\mathrm{LAPC}_\lambda(\Phi) = \min_{L, P_1, \ldots, P_T} \left[\lambda |L| + \sum_{t=1}^{T} |P_t \mid L| \right], \tag{2}$$

with $\lambda \geq 1$. Thus neither a circular declaration nor an unpriced advice oracle is allowed. The factor λ prices the design and maintenance of shared structure. A richer version can price names, dependency fragility, retrieval, and human validation separately. An online version charges when each declaration is introduced and compares cumulative cost with the best hindsight library.

Let $C(\varphi)$ be the shortest proof of one target without a new library. Because L may be empty, $\mathrm{LAPC}_\lambda(\Phi) \leq \sum_t C(\varphi_t)$. The gap

$$G_\lambda(\Phi) = \sum_t C(\varphi_t) - \mathrm{LAPC}_\lambda(\Phi) \tag{3}$$

is the compression attributable to shared mathematical infrastructure at the chosen interface price. At $\lambda = 1$ the objective is a restricted two-part description length: library plus residual certificates. It is not Kolmogorov complexity; the language is fixed to checked Lean artifacts and retrieval cost can be charged explicitly.

Even a syntactic shadow of this objective is computationally hard. If certificates are strings and library entries are restricted to acyclic macros, minimizing library plus residual size becomes the smallest-grammar problem, which is NP-hard and hard to approximate closely (Charikar et al., 2005). Typed declarations add validity, elaboration, retrieval, and semantic equivalence to that selection problem. Thus a long-horizon agent must solve not only proof search but a difficult representation-design problem about which repeated structure deserves a boundary.

The break-even law is immediate but useful. Ignoring interactions, a declaration of charged size b that saves s symbols in each of k residual proofs pays exactly when $ks > \lambda b$; with temporal discounting, ks becomes $\sum_i \gamma^{d_i} s_i$. The amortization horizon is therefore not a metaphor but the point at which checked shared structure beats repeated local construction.

Equation (2) changes what counts as an optimal proof. A declaration that lengthens the first episode may lower total cost if it compresses later proofs. A generic helper that is short but impossible to retrieve may fail to pay back its interface cost. A benchmark of independent theorems with a reset context implicitly sets future reuse to zero and selects a compile-first policy. Even pass@k with enormous search budgets remains a measure of finding current certificates, not of learning a mathematical language in which later certificates become cheap.

The connection to circuit sharing and extension variables is instructive but limited. Naming a repeated subexpression turns tree accounting into DAG accounting; introducing a reusable theorem resembles adding a subroutine or grammar rule. Yet Lean claims are dependent types, tactics alter elaboration, definitions compute, and public theorem statements carry semantic and maintenance costs. We do not claim an equivalence with Extended Frege. The robust lesson is accounting level: single-certificate complexity cannot see investments whose return lies across certificates.

This produces new empirical questions at the boundary of complexity and cognition. How large can online library regret be for a bounded-context agent? Which theorem orderings permit discovery of the hindsight-optimal abstractions? Does a human-readable interface impose only polynomial overhead on a machine-efficient lemma basis, or can correspondence itself be exponentially costly? DeDeo's "hard proofs" dilemma suggests that mathematical culture may be understood as a selection process over theorem streams and representations, concentrating attention where useful proofs and reasons coincide (DeDeo, 2024b). AI changes both the search distribution and the consumers for whom that coincidence must hold.

7 What the result does and does not establish

The evidence supports a narrow, structural claim. In this paired corpus, the AI track uses more source-level local state; more of it is visibly unreferenced, more of it vanishes from the final term's dependency structure, and its names are more generic. Conditional source duration does not robustly separate the tracks. It does not support a claim that machine proof terms lack sharing: retained binders are multiply used at the same rate. Nor does it show that human artifacts are globally reusable, since our primary analysis ends at the target declaration. The distinction between episode and persistent interface remains to be tested on theorem sequences.

Several limitations are consequential. First, source provenance is observational. Second, the lexical parser recognizes named `have`, not every `let`, `suffices`, anonymous claim, or tactic-generated subgoal. Third, the production-snapshot census is complete, but the independent current-snapshot sensitivity leaves 298 of 312 pairs; version attrition can select on style. Fourth, binder occurrence is syntactic and can be amplified massively by automation. Fifth, the main surprisal model may have seen related text, while a source-held-out bigram weakens the content-only contrast. The boundary effect is exploratory. Sixth, meaningful names and long spans are only proxies for explanatory value. Interactive links from written proof steps to Lean state have improved comprehension in a small reader study, but our boundary measures still require a direct experiment (Kambhamettu et al., 2026).

The representation audit adds one correction. An earlier root-term extractor globally interned raw de Bruijn indices from unrelated scopes, creating false reuse paths. Scope-specific identities remove the error, and a corrected 298-pair rerun finds no robust authorship effect in either the fixed-10 reuse-tail exponent (-0.002, source-cluster interval -0.016 to 0.013) or maximum outdegree (median difference zero, -3 to 0). The binder-use traversal here is independently scope-aware; details and regression construction appear in the appendix.

Three observations would falsify or sharply narrow the horizon account. The claim fails if an objective-matched human–AI experiment retains the same difference; if downstream reward does not produce longer-lived machine interfaces; or if the proposed interface measures fail to predict future proof cost, maintenance, or reader comprehension. Conversely, a machine system that builds a durable, intelligible library is not an exception to be explained away. It is the decisive prediction. Ultimately, a proof is a wager on its next use: evaluation should measure whether proving today changes tomorrow's cost and intelligibility.

References

AI-MO. NuminaMath-LEAN: A large-scale dataset of competition problems formalized in Lean 4. Hugging Face dataset, https://huggingface.co/datasets/AI-MO/NuminaMath-LEAN, 2025.

Claire Augusta Bergey and Simon DeDeo. From "um" to "yeah": Producing, predicting, and regulating information flow in human conversation, 2024. arXiv:2403.08890.

Moses Charikar, Eric Lehman, Ding Liu, Rina Panigrahy, Manoj Prabhakaran, Amit Sahai, and Abhi Shelat. The smallest grammar problem. *IEEE Transactions on Information Theory*, 51(7):2554–2576, 2005. doi: 10.1109/TIT.2005.850116.

Katherine M. Collins, Simon Frieder, Jonas Bayer, Jacob Loader, Jeck Lim, Peiyang Song, Fabian Zaiser, Lexin Zhou, Shanda Li, Sam Looi, Joshua B. Tenenbaum, Umang Bhatt, Adrian Weller, Jose Hernandez-Orallo, Cameron E. Freer, Valerie Chen, and Ilia Sucholutsky. Characterizing initial human–ai proof formalization workflows, 2026. arXiv:2606.04273.

Stephen A. Cook and Robert A. Reckhow. The relative efficiency of propositional proof systems. *Journal of Symbolic Logic*, 44(1):36–50, 1979. doi: 10.2307/2273702.

Simon DeDeo. Alephzero and mathematical experience. *Bulletin of the American Mathematical Society*, 61(3): 375–386, 2024a. doi: 10.1090/bull/1824.

Simon DeDeo. Hard proofs and good reasons, 2024b. arXiv:2410.18994.

Simon DeDeo and Eamon Duede. A correspondence problem for mathematical proof, 2026. arXiv:2603.13680.

Kosta Došen. Identity of proofs based on normalization and generality, 2004. arXiv:math/0208094.

Catarina Dutilh Novaes. A dialogical conception of explanation in mathematical proofs. In Paul Ernest, editor, *The Philosophy of Mathematics Education Today*, pages 81–98. Springer, 2018. doi: 10.1007/978-3-319-77760-3_5.

Yiming Fu, Peixuan Liu, Zichen Wang, and Kun Yuan. Proof-refactor: Refactoring generated formal proofs into modular artifacts, 2026. arXiv:2606.03743.

Guoxiong Gao, Zeming Sun, Jiedong Jiang, Yutong Wang, Jingda Xu, Peihao Wu, Bryan Dai, and Bin Dong. LeanSearch v2: Global premise retrieval for Lean 4 theorem proving, 2026. arXiv:2605.13137.

Cezary Kaliszyk and Josef Urban. Learning-assisted theorem proving with millions of lemmas. In *International Symposium on Symbolic and Numeric Algorithms for Scientific Computing*, pages 378–384, 2014. arXiv:1402.3578.

Hita Kambhamettu, Will Crichton, Sean Welleck, Harrison Goldstein, and Andrew Head. Making written theorems explorable by grounding them in formal representations, 2026. arXiv:2604.02598.

Gülce Kardeş and Simon DeDeo. Explaining necessary truths. In *Proceedings of the Annual Meeting of the Cognitive Science Society*, volume 47, 2025. arXiv:2502.11251.

David Kirsh and Paul Maglio. On distinguishing epistemic from pragmatic action. *Cognitive Science*, 18(4): 513–549, 1994. doi: 10.1207/s15516709cog1804_1.

Jan Krajíček. *Bounded Arithmetic, Propositional Logic and Complexity Theory*. Cambridge University Press, 1995. doi: 10.1017/CBO9780511529948.

Lean Developers. *The Lean Language Reference: The Type System and Propositions*, 2026. URL https://lean-lang.org/doc/reference/latest/The-Type-System/.

Yong Lin, Shange Tang, Bohan Lyu, Ziran Yang, Jui-Hui Chung, Haoyu Zhao, Lai Jiang, Yihan Geng, Jiawei Ge, Jingruo Sun, Jiayun Wu, Jiri Gesi, Ximing Lu, David Acuna, Kaiyu Yang, Hongzhou Lin, Yejin Choi, Danqi Chen, Sanjeev Arora, and Chi Jin. Goedel-prover-v2: Scaling formal theorem proving with scaffolded data synthesis and self-correction, 2025. arXiv:2508.03613.

Øystein Linnebo. Generality explained. *The Journal of Philosophy*, 119(7):349–379, 2022. doi: 10.5840/jphil2022119725.

Jialin Lu, Soonho Kong, Rodrigo Stehling, Kaiyu Yang, Zhangyang Wang, Weiran Sun, and Wuyang Chen. Lean refactor: Multi-objective controllable proof optimization via agentic strategy search, 2026. arXiv:2605.20244.

Hongqin Lyu, Junxing Dong, Yonghao Wang, Zhiteng Chao, Tiancheng Wang, and Huawei Li. Rtl2lean: Automated RTL-to-Lean translation with hierarchical theorem generation and lemma reuse, 2026. arXiv:2607.16855.

Yehuda Rav. Why do we prove theorems? *Philosophia Mathematica*, 7(1):5–41, 1999.

Claude E. Shannon. Coding theorems for a discrete source with a fidelity criterion. In *IRE National Convention Record*, volume 4, pages 142–163, 1959.

Scott Viteri and Simon DeDeo. Epistemic phase transitions in mathematical proofs. *Cognition*, 225:105120, 2022. doi: 10.1016/j.cognition.2022.105120.

Haiming Wang, Huajian Xin, Chuanyang Zheng, Lin Li, Zhengying Liu, Qingxing Cao, Yinya Huang, Jing Xiong, Han Shi, Enze Xie, Jian Yin, Zhenguo Li, Heng Liao, and Xiaodan Liang. LEGO-Prover: Neural theorem proving with growing libraries, 2023. arXiv:2310.00656.

Zachary Wojtowicz and Simon DeDeo. Undermining mental proof: How AI can make cooperation harder by making thinking easier. *Proceedings of the AAAI Conference on Artificial Intelligence*, 39(2):1592–1600, 2025. doi: 10.1609/aaai.v39i2.32151.

Ziyi Yang, Wenji Fang, Chen Chen, Zhiyao Xie, and Hongce Zhang. CircuitProver: Agentic Lean 4 theorem proving with reusable circuit proof library for hardware verification, 2026. arXiv:2607.27259.

Jiajie Zhang and Donald A. Norman. Representations in distributed cognitive tasks. *Cognitive Science*, 18(1):87–122, 1994. doi: 10.1207/s15516709cog1801_3.

Youyuan Zhang, Jialiang Sun, Hangrui Bi, Chuqin Geng, Wenjie Ma, Zhaoyu Li, and Xujie Si. DreamProver: Evolving transferable lemma libraries via a wake–sleep theorem-proving agent, 2026a. arXiv:2604.26311.

Yuanhe Zhang, Yuekai Sun, Taiji Suzuki, Jason D. Lee, and Fanghui Liu. LeanMarathon: Toward reliable AI co-mathematicians through long-horizon Lean autoformalization, 2026b. arXiv:2606.05400.

Jin Peng Zhou, Yuhuai Wu, Qiyang Li, and Roger Grosse. REFACTOR: Learning to extract theorems from proofs, 2024. arXiv:2402.17032.

A Reproducibility and robustness

All reported paired source results are produced by `code/paired_horizon.py`; scope-aware term use by `code/extract_binder_use.py` and `code/analyze_binder_use.py`; and surprisal by `code/prepare_surprisal.py`, `code/token_surprisal.cpp`, `code/ngram_surprisal.py`, and `code/analyze_surprisal.py`. Machine-readable summaries live in `results/horizon/`. Seeds and inclusion rules are serialized with the estimates.

Source intervals resample 12 source groups with replacement and recompute pooled claim ratios. Every source group has lower explicit uses per AI claim and higher AI zero-uptake; 11 of 12 have lower AI long-reach share. The production term alignment's unique-name subset contains 9,477 human and 15,058 AI claims; zero-use shares are 9.5% and 21.7%, respectively. The surprisal boundary contrast is present for four-, eight-, and more weakly 16-token windows, and absent at 32. Excluding the complete claim header preserves the four- and eight-token effects. In the leave-one-source-out bigram, which excludes every document in the held-out proof's coarse source group from training, the raw eight-token difference remains ($p = .017$) but content-only evidence weakens ($p = .051$).

The adoption–duration decomposition guards against a changing temporal ruler. For every adopted claim, token reach is the number of lexical tokens from the end of its header to its last explicit reference. Opportunity-normalized reach divides crossed later claim boundaries by the number available before the same-name stopping point and excludes declarations with no later boundary. Human token reach and normalized reach are higher in all 12 source groups. In contrast, the conditional chance of crossing at least one available boundary is 75.1%/75.2% (AI-minus-human source-cluster interval −.013 to .011). In the length-matched subset, both conditional-duration intervals cross zero; under equal positive claim counts, normalized duration is 50.9%/53.8%. The claim-density ruler alone would conceal both the null result and the instability of the pooled direction.

The local functional check on the family classifier holds separately for its two equivalent spellings. Within eligible proofs, the multi-use uplift for a pre-type-binder family is 17.2 points human and 25.5 points AI; for a leading-universal family it is 25.6 and 33.0 points. All four source-cluster intervals exclude zero. Thus the result is not created by unioning a human-favored and an AI-favored surface form.

Claim position is a second mechanical alternative: an early family simply has more remaining text in which to be referenced. Within each proof we therefore use minimum-cost one-to-one matching on claim index normalized by the proof's final claim index. With no caliper this yields 637 human and 478 AI claim pairs; with an absolute position-gap caliper of .25, it yields 433 and 409 pairs in 269 and 226 proofs. In the calipered comparison, family-minus-instance adoption is 10.2 points human and 15.8 AI, while multiple-reference uplift is 20.8 and 31.0 points. All intervals exclude zero. The explicit-reference count itself is less stable for humans, but the two binary functional outcomes are not explained by simple exposure time.

We also attempted a stronger test of the naming interpretation. In 230 pairs with two to eight unique, explicitly typed claims per proof, each naturalized name had to retrieve its own type from the other types in that proof using Goedel-Prover mean embeddings. Mean percentile rank is .528 human and .518 AI (AI-minus-human source-cluster interval −.022 to .009); chance-adjusted top-one accuracy is likewise unresolved. Last-token pooling gives the same null direction, and among 57 equal-claim-count pairs mean-pooling top-one accuracy is 40.9% human and 40.7% AI. Because this causal model was not trained as a contrastive encoder, even a positive result would have been exploratory. The null is sufficient for the present inference: vocabulary size and generic-form frequency do not establish that human names are semantically better retrieval keys.

The preselected 312-pair current-version sample closely preserves the source contrasts. Human/AI claim densities are 1.98/3.25 per 100 tokens in the sample versus 2.01/3.18 in the full corpus; explicit uses per claim are 1.38/0.84 versus 1.44/0.87; and zero-uptake shares are 22.3%/35.7% versus 22.2%/36.3%. All 12 source groups remain represented. Current-version attrition reduces this to 298 complete binder-use pairs but changes the profile little.

Paired does not mean independent: 181 structured proof values are token-identical and 230 have whitespace-token similarity at least .90 after comments and strings are removed. These pairs attenuate rather than create the result. Of the 181 exact value matches, 180 also have identical source bodies, so they are duplicate mass rather than a source/certificate natural experiment. Excluding all 230 leaves 3,400 pairs; explicit uses per claim are 1.42/0.85, zero-uptake shares 22.9%/36.7%, and placeholder shares 59.0%/89.5%.

The source direction also survives a coarse automation sensitivity. For each of `linarith`, `nlinarith`, `norm_num`, `omega`, `ring`, and `simp`, we separately retain pairs in which both sides invoke the tactic and pairs in which neither does. Across all 12 strata, AI claims have fewer explicit references per claim and a higher zero-uptake share. This does not equate full tactic workflows, but rules out any one of these common families as

a sufficient explanation.

B Exact counting without tree expansion

The production extractor never materializes the expanded proof tree. For each expression node e, it computes a summary $S(e) = (N_e, O_e, B_e)$: the number N_e of nodes in the fully expanded syntax tree, a finite map O_e from free de Bruijn depth to occurrence count, and the ordered local binders decoded below e. Application and constructor nodes add child summaries once per child occurrence. Entering a binder records $O_e(0)$ as its use count, drops that coordinate, and shifts the remaining coordinates down by one. Because N_e is an arbitrary-precision natural, shared subexpressions can contribute exponentially without expansion. An explicit work stack makes both the semantic summary and raw-tree recurrence stack safe.

Memoization requires one subtle restriction. Lean expression hashes are insensitive to some alpha-renaming, while this audit must recover the source-facing binder names. Caching a binder-bearing summary can therefore transplant a hygienic name between alpha-equivalent scopes. The production traversal caches only summaries with no decoded binders; de Bruijn occurrence maps remain sufficient for scope-correct composition. Every extraction records the template, toolchain, manifest, and corpus hashes.

We checked equivalence rather than trusting the optimization. A prespecified audit unions the first 50 lexicographic legacy-success pairs, the 50 largest legacy-success roots, and ten seeded random pairs from each of 12 source groups. This yields 215 distinct pairs (430 tasks), with a largest legacy root of 97,350,243 nodes. All 430 memoized records equal the recursive legacy records after replacing only the temporary-work-tag substring inside Lean-generated hygienic names; 427 already agree byte-for-byte as parsed JSON. The same canonical comparison over all 7,186 tasks completed by the legacy traversal is exact, and the memoized traversal additionally recovers all 74 legacy timeouts. The audit selection, mismatches, hashes, and summary are serialized in `results/horizon/binder_memo_equivalence.json` and `results/horizon/binder_memo_full_equivalence.json`.

The recurrence example in the main text is the largest representational extreme in this complete census. Fully expanded tree counts reach 11 decimal digits on the human side and 535 on the AI side; four AI tasks exceed 20 digits and three exceed 100, versus none for humans. These counts are useful precisely because they expose the formula represented by a shared expression DAG. They are not heap size, checking time, semantic proof complexity, or evidence that Lean constructs the tree.

C Executable representation regressions

Three small Lean regressions make the invariance corrections independently checkable. First, `code/test_legacy_binder.py` constructs the same local family once with pre-type binders and once with an explicit universal type. Under Lean 4.15 both elaborate to a named `letFun` binder used once. The source classifier must place both in its generalized class.

Second, `code/test_binder_toolchain_shift.py` elaborates the same real corpus theorem under the production and current snapshots. Its source claim `h1` is a decoded `have_fun` in Lean 4.15 and a nondependent `let` in Lean 4.33, with one occurrence in both. This is why raw node kind cannot define retention across versions.

Third, `code/test_certificate_representation.py` builds a well-typed `Nat` expression e_{n+1} = `let x :=` e_n`; x + x`, invokes Lean's zeta reducer, and asks Lean to infer both types. At depth 14 the raw and reduced expressions both have type `Nat`, their types are definitionally equal, and their measured sizes are 99 and 65,533 nodes. These are regressions, not benchmarks: their purpose is to prevent later analyses from silently treating a representation choice as a fact about reasoning.

D Why the old network result is not the new result

The prior project replicated Viteri and DeDeo's fixed-tail power-law results and modeled epistemic phase transitions across Coq and Lean. Those are properties of declared network constructions. They do not supply a representation-invariant authorship signature. Tail exponents move when the minimum degree or expansion boundary moves; human–AI directions can reverse under proof-value normalization; and modeled theorem belief is close to saturation in both tracks.

The scope audit strengthens the reason for restraint. A de Bruijn index identifies a binder by distance in its local expression, not by a global name. Interning equal raw indices across scopes turns distinct variables into one graph node. The corrected traversal carries a scope environment and replaces each entered binder with a fresh free-variable identity before hashing expressions. The regression proof has two independent identity lambdas; the old representation produces one shared variable leaf, while the corrected representation produces two. None of the main analyses in this paper uses a raw-term reuse tail.

The scope-correct rerun elaborates 302 of 312 tasks on each side; applying the independent binder- task status filter leaves 298 complete pairs. AI terms are slightly larger (median paired node difference 88.5; source-cluster interval 0–243) and have a slightly larger visit-to-unique-node ratio (difference 0.0068; 0.0023–0.0112). But the proposed authorship signal in high-outdegree reuse remains absent: maximum outdegree and the fixed-10 tail exponent have intervals crossing zero. These graph quantities remain representation-specific diagnostics, not measures of cognitive abstraction.

E Rate–distortion form of the horizon principle

Let an episode $E \sim p(e)$ include the derivation history available at consolidation time, let $U \sim q_\gamma$ be a future use drawn from a horizon-weighted task distribution, and let an interface encoder choose $Z \sim p(z \mid E)$. For consumer c, let $d_c(U; E, Z) \geq 0$ charge downstream search, repair, retrieval failure, or loss of the ability to track the proof idea, and define horizon-weighted distortion

$$D_c(E, Z) = \mathbb{E}_{U \sim q_\gamma}[d_c(U; E, Z)].$$

A consumer-specific interface frontier is

$$R_c(\delta) = \min_{p(z|E):\ \mathbb{E}D_c(E,Z) \leq \delta} I(E; Z).$$

For a deterministic encoder, the rate is the information retained in the chosen interface. A short horizon changes q_γ toward the immediate goal, making positional scratch state cheap and distant transfer nearly irrelevant. A different consumer changes D_c: an opaque lemma may have low distortion for a prover with effective retrieval and high distortion for a reader who cannot connect it to the proof idea.

This formulation clarifies three claims. First, consumer-invariance is the zero-distortion case: artifacts in the same response class must be indistinguishable to a statistic that purports to measure their function for that consumer. Second, generality increases the support of uses over which a code can pay, while naming can lower retrieval loss without shortening the certificate. Third, equation (2) is a checked, finite-stream code-length analogue of the same trade-off: library size is rate, and residual proofs are downstream distortion measured in certificate cost. The human–AI question is consequently not who compresses, but which future-use distribution and which distortion function govern the compression.

F A proposed causal benchmark

A strong test would sample theorem sequences, not independent goals. Randomize capable agents to a compile-first condition or a library condition. Both receive the same base library, target order, Lean version, and inference budget. The library condition is charged for current proof cost but rewarded for reductions in later search, version repair, and blinded reader retrieval. Public declarations must be generated before future targets are revealed to prevent hindsight leakage.

Primary outcomes should include cumulative checked-token or search cost, online regret against a hindsight refactoring, survival of generated APIs under version changes, cross-target citations, dead local binders, and delayed human comprehension. A swap condition—humans under disposable single-goal incentives and AI under persistent-library incentives—would separate constructor from horizon. The strongest result for the present theory would be a crossover interaction rather than a persistent main effect of provenance.